

FactWorld: Non-Abelian State — A Learnability Study of the Composition Gap

Ian Barber — June 2026

Abstract

Sequence models for agentic workloads must track evolving state and recall facts in a single pass, often past the training horizon, yet *why their composition fails* has stayed unclear. On FactWorld, an oracle-validated instrument, we localize the recall $\times$ state-tracking composition gap and map when it becomes learnable, holding the architecture (a GatedDeltaProduct recurrent hybrid) and world fixed while varying the fact source, the density of intermediate state supervision, the training schedule, and model scale. Two of these are orthogonal axes: *where the fact lives* (in weights or in the prompt) and *how the state evolves* (abelian or non-abelian). The first is *free* — given a resolved pointer, recalling a fact from the weights is as reliable as copying it from the prompt ($P(\text{value} \mid \text{holder}) = 1.0$) — so the gap is the non-abelian state leg, not the fact source. That leg forms only under near-dense process supervision: a sharp threshold, a checkpoint roughly every other step with chance below, that an agent’s sparse outcome signal cannot supply and that outcome-reward reinforcement learning cannot climb either. Weaning to no supervision internalizes the circuit only at the trained length and never beyond — a dissociation that scaling to 357M (7.9 $\times$ past the companion model) leaves untouched, the largest *fully-converged* model still flooring past its trained horizon, which places the wall in learnability, not capacity. What *does* move it is the training-length distribution: a sufficient density of target-length examples unlocks extrapolation (a threshold near one-fifth of training, then a plateau, with a soft cap on the maximum trained length), and relatedly a curriculum that *progressively* grows the trained sequence length lifts internalized, no-scratchpad accuracy at 4 $\times$ the training horizon from the 0.20 chance floor to **0.94**, whereas an explicit token-level state summary stays at floor.

Code & data. Every number maps to one script under `followups/parametric-recall/` in github.com/ianbarber/factworld; the data/oracle layer is pure-stdlib, training requires a CUDA GPU.

1. Introduction

The companion FactWorld paper (Barber, 2026) established that the recall $\times$ state-tracking composite, while it floors at small scale, becomes learnable for a product-structured recurrent hybrid, and is learned as a conditional resolve-then-recall pipeline rather than two parallel legs. Two questions follow immediately, and both matter for agentic and code-like workloads, where a model must maintain evolving state over a long horizon and answer sparse queries against facts that live in-context or in weights.

First, *which leg is the bottleneck* — the state-tracking, or the recall, or specifically the parametric (in-weights) recall that an agent must do without the fact in front of it? Second, *is the composite reachable under realistic supervision* — can a model learn to maintain the latent state from the sparse, outcome-level signal an agent gets, and run that computation past the horizon it was trained on?

We answer both on the same oracle-validated instrument, holding the architecture (a GatedDeltaProduct hybrid) and world fixed and varying only the fact source, the supervision density, the training schedule, and the model scale. The contributions:

1. **The bottleneck is the state leg; recall is free.** A decomposition shows parametric and in-context recall are indistinguishable *in-distribution* given a resolved pointer (and parametric extrapolates at least as well at length; §3–§4). This corrects a natural reading of the companion paper — that *parametric* recall is the obstacle — and relocates the gap to non-abelian state-tracking.
2. **Process supervision is a cliff.** Dense per-step state supervision solves the composite; a sparsity sweep shows the circuit forms only down to a checkpoint every other step and collapses to chance below that (§4). Sparse/answer-only supervision — the agentic regime — gives no traction.

3. **An internalization/horizon dissociation, not relieved by scale.** Weaning internalizes the circuit at the trained length (on a minority of seeds) but never beyond; externalized tracking extrapolates, internalized tracking does not; and the wall stays at floor across the full scale ladder to 357M (7.9× past the companion model) — the largest *fully-converged* model still floors past its horizon (§5).
4. **The training-length distribution moves the wall that scale cannot.** A sufficient density of target-length examples — a threshold near one-fifth of training, with a soft cap on the maximum trained length — unlocks internalized extrapolation, and a progressive horizon-extension curriculum lifts internalized 4× accuracy from floor to 0.94; a compressed token-level state summary, by contrast, stays at floor (§6).

2. Setup

We reuse FactWorld’s non-abelian composite: a stream of `swap_role/cycle_roles` events permutes which agent holds each of $k = 5$ roles (the canonical S_5 word problem), each agent carries a static fact, and the query “*what is a0 of the holder of role r?*” requires resolving the role to its current agent (state) and then recalling that agent’s fact (recall). The role→agent resolution is computed by the symbolic oracle, so the headline label is correct by construction.

The composite crosses **two orthogonal axes**, which we vary independently:

- **Where the fact lives** — *parametric* (the agent→fact map is fixed across all training and *not* rendered in the prompt, so recall must come from the weights) versus *in-context* (the map is in the prompt). This is the **recall** leg.
- **How the state evolves** — *abelian* (object hand-offs, last-write-wins) versus *non-abelian* (the role permutations above, which do not commute). This is the **state** leg, and it is the source of the non-abelian difficulty — independent of where the fact lives.

Keeping these axes separate is the paper’s first move: the difficulty is the state axis, and §3 shows the fact axis is free. The fact source is **parametric** unless stated otherwise. The model is the product-structured GatedDeltaProduct hybrid (`gdp_hybrid`); the floor is $1/k = 0.20$.

Training vs. evaluation. Training is standard next-token prediction on sequences that interleave the oracle’s exact state after each event, so intermediate state is *teacher-forced* into the training stream. At evaluation the model is **free-running**: it emits its own intermediate-state tokens and final answer with no oracle signal, and we score the final answer (end-to-end accuracy). Where stated we also report *teacher-forced per-step* state accuracy — a single forward pass over the true trace, scoring each state slot — which isolates whether the per-step transition was learned from whether free-running errors compound.

3. The bottleneck is the state leg, not recall

A four-rung ladder localizes the break. Literal-key parametric recall is trivial (1.00). A pointer resolved by an *abelian* binding (last-write-wins) dereferences the parametric memory with no scratchpad needed (0.83 at the train length, 0.57 at 4×), and this is robust to scoring: position-strict and value-scan metrics agree to the digit (the answer is a single token at the answer position), so it is not a scoring artifact. (This abelian configuration is easier than the companion paper’s flagship composite, which couples parametric recall with a harder binding at longer lengths and floors there; we report the abelian dereference on its own terms and make no correction claim about that composite.) The composite floors only when the pointer is resolved by *non-abelian* state, and chain-of-thought that emits *only the final holder* — with no per-step state supervision — does not rescue it: it supervises output order, not the intermediate state.

The decisive evidence that **recall is not the bottleneck** is the dense-supervision result of §4: with a per-step state trace, the **parametric** composite solves end-to-end and extrapolates *at least as well as* in-context (0.95 ± 0.06 vs 0.55 ± 0.37 at 4×). Recalling from weights is no harder than copying from the prompt once the pointer is resolved.

A decomposition corroborates the mechanism. Splitting the strict score into a state leg (holder correct) and a recall leg (value correct), run for in-context and parametric arms on identical chains, gives **identical** results — both at floor on the holder, but with $P(\text{value} \mid \text{holder}) = 1.0$ and, on holder-wrong examples, routing = 1.0 (the model emits the value of whatever agent it resolved). The *independent* failure is the holder; value-given-holder is perfect in both arms. Both legs sit at chance in this control, so it confirms the *mechanism*; the *magnitude* (parametric solves and extrapolates) comes from the dense-supervision result of §4.

Recall is free: in-distribution, parametric and in-context recall are interchangeable given a resolved pointer (and at length parametric is the stronger of the two, §4). The composition gap is a state-tracking-under-supervision problem.

3.1 Abelian is a real length-general circuit; non-abelian has no shortcut to be one

A length-decay probe to $16\times$ the training length sharpens *why* the non-abelian leg, specifically, is the wall. Training at the short envelope (max length 16) and evaluating to length 256, with the query stated **at the end** (the model reads the whole history, then answers), the abelian dereference settles to a stable plateau (~ 0.55 , well above the 0.20 floor) all the way out to $16\times$ the trained length — a genuinely **length-general register circuit** — while the non-abelian composite stays at floor at every length without per-step supervision. The difference is structural: last-write-wins admits a **read-then-lookup** solution (scan the history for the last write to the queried object), which is length-invariant; the non-abelian S_5 composition (NC^1 , not TC^0) admits **no** such scan-and-lookup, so it can only be solved by carrying the running permutation in recurrent state. When even the abelian task is forced onto that **online-carry** path — the query stated up front, so the running answer must be maintained step by step — it too cliffs past $\sim 1.5\times$ the trained length, a recurrent-state-calibration limit: the state distribution beyond the trained depth is never visited in training (the “unexplored states” hypothesis of Buitrago Ruiz & Gu, 2025). So the non-abelian leg is the wall not merely because it is harder, but because its *only* available solution path is the length-bounded one; the abelian leg escapes through a length-general read-then-lookup that the non-abelian leg cannot use. This last point is not the end of the story, though: the online-carry path *can* be calibrated. Driving the recurrent state to the depths seen only at test time — via an unlabelled burn-in that exposes the state distribution at length — *fails* when done from scratch (it floors and even degrades in-distribution training, the circuit never forms), but applied as a **post-training intervention to an already-formed circuit** (in the spirit of Buitrago Ruiz & Gu, 2025) it succeeds on a minority of seeds, extending internalised, no-scratchpad accuracy from the floor to **0.99 at $4\times$ and 0.86 at $8\times$ the trained length, with no labels at length** — a seed-fragile existence proof that a length-general non-abelian circuit is reachable, and direct confirmation of the unexplored-states diagnosis. So the non-abelian leg has no length-general *shortcut*, but its length-bounded online-carry solution can be made to extrapolate by calibrating the state distribution post-hoc; making that lever reliable (it fires on roughly a third of seeds) is the open problem.

4. Process supervision is a cliff

Interleaving dense per-step state supervision (the oracle’s holder after each event) into the stream solves the composite end-to-end at the trained length (1.00) for both fact sources, and the parametric variant extrapolates to $4\times$ *better* than in-context (0.95 vs 0.55; a fixed weight lookup adds no per-step ambiguity, while in-context copy over a long trace has more to go wrong).

The question is how dense that supervision must be. Supervising the holder every K events (always keeping the final, so recall stays keyed) traces a sharp threshold:

K (supervise every K events)	L16 (in-dist.)	L64 ($4\times$)
1 (dense)	1.00	0.78
2 (every other)	0.98	0.29
4 (every 4th)	0.19 (floor)	0.19
∞ (answer-only)	0.19	0.20

It is a **cliff**: the circuit forms in-distribution only down to a checkpoint every other step; at $K = 4$ even accuracy *at the labelled slots* is at floor, so the recurrence fails to discover the permutation computation at all. Length extrapolation is stricter still — only fully dense supervision ($K = 1$) extrapolates. Sparse, outcome-level supervision gives zero traction.

The cliff is not specific to static supervised learning. Agentic models are trained with reinforcement learning (RL), whose credit assignment differs from next-token supervised fine-tuning (SFT) — exploration and advantage-weighting can in principle route a sparse terminal reward back to intermediate computation. We tested it: starting from an answer-only-SFT model (at the floor) and running Group Relative Policy Optimization (GRPO; Shao et al., 2024) with a 0/1 reward on the composite answer and a free scratchpad, the reward stays pinned at chance across 5,000 steps (≈ 0.20 ; holder-resolution rate 0.00 throughout) and the policy never exceeds the SFT floor.

The barrier is *structural*, not merely empirical. Non-abelian state-tracking is serial and admits no partial credit: a depth- n answer is correct only if the entire n -step composition is correct, so the probability of sampling a correct rollout by chance is on the order of k^{-n} . Reward variance therefore never rises above floor, advantages are noise, and policy gradient has nothing to bootstrap from — exploration is intractable, not just empirically null. “Process supervision is required” is thus robust across training paradigms: static SFT *and* outcome-reward RL both fail without it. (One architecture, one RL recipe; reward shaping or a length curriculum could change this — see §9 — but vanilla outcome reward does not climb the cliff.)

4.1 The same cliff in code-execution clothing

A reviewer’s natural worry is that the S_5 word problem is a synthetic abstraction. To check that the wall is non-abelian *composition* and not our role-permutation surface form, we re-render the **identical** dynamics as a variable-swap execution trace, in the surface grammar of **Code World Model (CWM; Meta FAIR, 2025)** — a code LLM mid-trained on interleaved per-line execution-state traces (§8). Variables hold values, *swap/cycle* ops permute them, the full program state is interleaved every K ops (a CWM-style observation), and the query asks a variable’s final value. This is a recognizable code-execution task: tracking a variable through a sequence of reassignments is the same non-shortcuttable composition.

The same density cliff appears, sharpest in length-extrapolation (L64): $0.97 \rightarrow 0.89 \rightarrow 0.50 \rightarrow 0.29 \rightarrow \text{floor}$ as snapshots thin from every-op to answer-only. The in-distribution threshold sits at *sparser* K than the role-rendered sweep, because a full-state snapshot carries more information per checkpoint than the single-role checkpoint of the §4 table — mechanism-consistent: richer process supervision needs lower density. (At $K = \infty$ the full final-state snapshot turns the task into “track to the end, emit the state, read off the variable,” which a minority of seeds solve and most floor.) The non-abelian wall thus reproduces directly in the code regime, and needs the same near-dense process supervision — evidence that the finding is about the computation, not the synthetic dressing.

5. Internalization, the horizon dissociation, and scale

Can a model trained densely keep the circuit when supervision is removed? Gradual scratchpad removal (annealed density) and a mixed-density control both **internalize** the circuit on some seeds — running with no scratchpad and clearing the floor (best seeds = 1.00 at the train length). Two findings qualify this. First, *order is not the lever*: mixed density $\geq$ annealed curriculum on every metric, so what helps is exposure to both densities, not the schedule. Second, and most consequentially, **internalization does not extrapolate** — answer-only accuracy at $4\times$ is at floor even for seeds that internalize perfectly at the trained length. This is a clean dissociation:

- **externalized** (scratchpad) state-tracking extrapolates in horizon but keeps the crutch;
- **internalized** (no scratchpad) works at the trained length but is brittle to horizon.

Length-generalization **or** no-scratchpad — not both.

Scaling the mixed recipe sharpens this: more parameters make internalization *reliable at the trained length* and improve externalized extrapolation, but the internalized horizon wall (answer-only L64) does not move. We push the ladder from 5.7M to **357M** — 7.9× past the companion paper’s ~45M model and to the 3090’s batch-32 memory ceiling — holding the recipe fixed (mixed density, lr 1e-3, 8000 steps); the smaller scales were additionally re-run learning-rate-tuned ($\in \{1e-3, 5e-4\}$) to rule out under-training, with a **70M control point** (a deliberate step past the ladder top, *not* a hardware limit — 70M trains in under 5 GB on the 24 GB card).

parameters	internalization in-range (L16)	answer-only L64 (the wall)
5.7M	0.67 (2 of 3 seeds)	0.23 (0 of 3)
18.5M	0.99 (3 of 3)	0.20 (0 of 3)
44.8M	0.95 (3 of 3)	0.22 (0 of 3)
70M	—	0.41 (2 of 2)
140M	1.00 (2 of 2)	0.42
268M	0.81 (under-trained)	0.21
357M	1.00 (2 of 2)	0.20 (floor)

The wall does not climb with width: answer-only L64 runs $0.23 \rightarrow 0.20 \rightarrow 0.22 \rightarrow 0.41 \rightarrow 0.42 \rightarrow 0.21 \rightarrow 0.20$ across the ladder. The only departure from floor is a weak, learning-rate-sensitive bump at 70M–140M (~0.42; at 70M, 0.41 at lr 1e-3 but 0.24 at lr 5e-4) that does *not* grow with further capacity. The decisive point is **357M**: it converges in-distribution on both seeds (L16 = 1.00) yet answer-only L64 sits exactly at the 0.20 floor — the largest model, with no under-training excuse, does not extrapolate at all. (At the fixed 8000-step budget the 268M runs sit at the edge of convergence — one seed reached only L16 = 0.63 — so that row is compute- as well as capacity-bounded, and we do not read its low L64 as a capacity effect; 357M, which *did* converge, carries the claim.) **Capacity is not the extrapolation lever.** There is principled reason for this: the number of distinguishable non-abelian state histories grows combinatorially (reachable configurations are elements of the symmetric group on the k roles, composed over the event sequence), so width — which buys parallel lookup and memorization — does not address the serial composition the task demands. What *does* move the wall is the training-length distribution (§6).

6. Moving the wall

With capacity (up to 357M) not relieving the wall, we test two candidate fixes, both motivated by reading a long session as a composition of short, well-supervised sub-sessions. We report per-seed counts ($n = 3$) and treat a fix as supported only when a majority of seeds clear the floor at the extended length, so a single lucky seed cannot be read as a fix.

Horizon-extension curriculum works in-range. The schedule is *progressive*: training admits longer sequences over the run ($8 \rightarrow 16 \rightarrow 32 \rightarrow 48 \rightarrow 64$, mixed density), not a static train-at-64. This lifts internalized, no-scratchpad tracking off the floor: answer-only accuracy is 1.00 at L16 and **0.94 (3 of 3 seeds) at L64**, where the same model trained at ≤ 16 sat at the 0.20 floor. The wall that scale could not move is moved by training the recurrence to run that many steps. Beyond the trained horizon the evidence is preliminary: at L128 ($2\times$ the maximum training length) accuracy is 0.48 (2 of 3 seeds above floor) — a hint of partial extrapolation, not an established result.

Token-level re-anchoring does not. Emitting a bounded full-state digest every C events and re-anchoring stays at floor at every cadence and is no better than the single-role control (L64/L128: full_C8 0.15/0.22, full_C16 0.20/0.19, single_C8 0.23/0.21; 0 of 3 seeds throughout). This is the cliff’s predicted consequence. The digest cadence is itself a supervision density — a digest every C events is supervision every C events — and at $C = 8$ and $C = 16$ it sits well inside the collapsed regime of the §4 cliff. So the model cannot learn to *generate* a correct digest to re-anchor on; at evaluation it feeds its own (often wrong) digests forward, and error

compounds across summaries exactly as across unsupervised steps. Externalizing the state into tokens re-imports the drift it was meant to bound.

The lesson. The horizon wall is closed by extending the *latent recurrence*’s trained horizon, not by adding explicit token-level state summaries. “Compose short sub-sessions” succeeds when each sub-session trains the recurrence to run longer, and fails when it asks the model to write and re-read its own state — because writing the state is itself the step it cannot yet do without supervision.

The internalized **cap tracks the max training length**: trained to length $L_{\max}$, the no-scratchpad circuit answers within $\sim[0, L_{\max}]$ and floors at $\sim 2 \times L_{\max}$ — so you extend reach by training the recurrence longer. The returns are noisy, though: in-range accuracy stays solvable but seed-fragile across $L_{\max}$ (0.37–0.82, high variance, no clean trend), because internalizing longer non-abelian chains is itself harder.

What distribution of training lengths unlocks the wall has a sharp structure. Holding capacity fixed (18.5M) and varying only the mix — short $\{4, 8, 16\}$ plus a fraction of target-length (L_{64}) examples — internalized L_{64} accuracy shows a **threshold and then a plateau**: a 5% long-fraction does nothing (floor), 20% lifts it to 0.66, and 50% adds nothing further (0.67). The critical fraction is in (5%, 20%]; above it, accuracy saturates at ~ 0.67 at this capacity, not at 1.0. Two refinements sharpen what “needs examples at length” means. The cap is **soft**: training only to length 32 ($\{16, 32\}$) still reaches $L_{64} = 0.37$ — partial extrapolation $2 \times$ past the maximum trained length, above the floor, not a hard wall. And **concentration beats coverage**: a uniform spread over $\{4 \dots 64\}$ reaches only 0.49 at L_{64} , *below* the concentrated short + 20%-target recipe (0.66), because density *at the target length* is more sample-efficient than breadth. So extrapolation is bought not by the mere presence of long examples but by a sufficient density of them at the target — the wall that capacity could not move (§5) yields to the training-length distribution. (L_{128} stays at floor under every mix: nothing trained at or near 128 reaches it, consistent with the soft cap decaying to chance by $\sim 2 \times$ the longest trained length.)

Supervision density (§4) and training horizon are **orthogonal levers, not a trade-off**. Re-running the density sweep at long-horizon training (lengths ≤ 64) leaves the cliff threshold unchanged — $K \geq 4$ floors whether trained at 16 or 64, so training at length does *not* let sparser supervision form the circuit — but at densities above threshold it sharply improves extrapolation ($K = 2$: $0.29 \rightarrow 0.98$ at $4 \times$). Density gates whether the circuit *forms*; horizon gates how far it *extrapolates* once formed.

7. What is established, and what is open

Two studies fix the shape of the result. Scaling the mixed recipe to 357M (§5) shows the internalized horizon wall does not move with capacity — the largest fully-converged model floors past its trained horizon. Varying the training-length distribution at fixed capacity (§6) shows the same wall *does* move when a sufficient density of target-length examples is present. Read together they say plainly what is true here, and what is not yet.

Established (on this instrument).

- *The non-abelian circuit is learnable in-distribution.* Trained to convergence, the model solves the composite at the trained length at every scale we ran ($L_{16} \approx 1.0$ from 18.5M to 357M). The in-distribution circuit is never the obstacle; the entire difficulty is out-of-distribution horizon.
- *Capacity is not the extrapolation lever.* Answer-only L_{64} does not climb with width across 44.8M→357M ($0.24 \rightarrow 0.41 \rightarrow 0.42 \rightarrow 0.21 \rightarrow 0.20$), and the fully-converged 357M model floors. Extrapolation failure is not a width or lookup-parallelism deficit.
- *The training-length distribution is the lever, with a threshold and a soft cap.* A sufficient density of target-length examples unlocks extrapolation (critical long-fraction in (5%, 20%], saturating ~ 0.67 at 18.5M); the reachable horizon is a soft cap on the maximum trained length; concentration at the target beats uniform coverage.
- *Density and horizon are orthogonal.* Supervision density gates whether the circuit *forms* (the §4 cliff, threshold $\approx K = 2$); the training-length distribution gates how far it *extrapolates* once formed.

Open (the frontier).

- *The threshold law.* We bracket the critical long-fraction to (5%, 20%] at one capacity and one target length, but not its shape, its dependence on the target horizon, or whether it shifts with capacity.
- *The capacity $\times$ length interaction.* The two levers are measured separately. Whether more capacity lowers the long-fraction threshold (cheaper extrapolation) or raises the reachable horizon for a fixed budget is unknown — the ~ 0.67 plateau at 18.5M, even with abundant long examples, hints the two must rise *together* past some frontier.
- *A predictive law.* Nothing yet predicts, for a target horizon, the (capacity, length-distribution) pair that reaches it — the practical question for deploying internalized state-tracking on long agentic sessions, and the natural next instrument-scale study.

8. Related work

Our results sit directly against Mozer, Siddiqui & Liu (2026), *The Topological Trouble With Transformers*, a position paper arguing that feedforward transformers have a *topological* limit on state tracking — state is pushed deeper into the layer stack each step until depth is exhausted — and that the field should refocus from externalized thought traces to implicit recurrent dynamics. It offers a taxonomy of recurrent/continuous-thought architectures and places DeltaProduct (Siems et al., 2025) and negative-eigenvalue DeltaNet (Grazzi et al., 2024) in its most-expressive cells: exactly our architecture.

Our follow-on is the empirical complement to that thesis. It is about *expressivity/constructability*, and states plainly that the log-depth state-tracking proof (Merrill & Sabharwal, 2025) “addresses only the constructability of solutions, not their learnability.” Our entire arc — the supervision cliff, internalization, the horizon dissociation, the scale check — is the *learnability* map it leaves open. Two specifics: (i) a recurrence-attention hybrid still hits our supervision cliff, complicating its expectation that such coupling dissolves the credit-assignment bottlenecks of recurrent training (we do not claim our layer-interleaved hybrid is exactly the step-coupled recurrence it has in mind); (ii) our scale result indicates the horizon wall is not an expressivity/depth limit that capacity relaxes (flat across the ladder to 357M, the largest fully-converged model still flooring), locating it in the learnability dimension the paper sets aside. Its “coarse-grained recurrence” direction is the token-level state summary we test — and find wanting — in §6.

The framing also connects to **execution-state world-modeling in code models**. Code World Model (CWM; Meta FAIR, 2025), introduced in §4.1, mid-trains a 32B model on interleaved per-line execution traces — predicting the program’s intermediate variable state, not just its tokens — and improves execution-reasoning benchmarks. This is process supervision of program world-state at frontier scale, and it is the *externalized* end of our axis: the model emits the trace. CWM does not isolate *which* state is hard (its per-line value snapshots are largely abelian last-write-wins, not the order-dependent composition we find is the wall) nor vary supervision *density* (it sits at the dense end). Our instrument isolates exactly those two — the non-abelian variable and the supervision-density cliff — and points past the emit-the-trace regime to the open problem of *internalizing* the world-model without writing the full trace. We render our non-abelian state-tracking task in CWM’s execution-trace surface grammar (§4.1) to check that the same wall appears in code-execution clothing, not only in the abstract S_5 word problem.

State-tracking expressivity and the product-recurrence mechanism are prior art (Liu et al., 2023; Grazzi et al., 2024; Siems et al., 2025; Merrill & Sabharwal, 2023/2025). The parametric-vs-in-context axis connects to where knowledge lives in weights (Geva et al., 2021; Meng et al., 2022) and to latent multi-hop reasoning (Biran et al., 2024; Yu et al., 2025). The internalization curriculum is gradual scratchpad removal — internalizing explicit reasoning into the weights (Deng et al., 2024).

9. Limitations

One architecture (`gdp_hybrid`), one world ($k = 5 S_5$); the cliff location ($K \approx 2$) is specific to this setup, and a different recipe or much larger scale could shift it. We addressed the under-training objection — the ladder runs

to 357M and the largest *fully-converged* model still floors (§5); at the fixed 8000-step budget the 268M runs sit at the edge of convergence, so we lean on the converged 357M point for the capacity claim. A capacity effect at far larger scale than 357M is not ruled out. Our RL evidence is a single recipe (outcome-only GRPO): process-reward models and search-guided exploration (e.g. PRMs, MCTS) inject intermediate signal or structure the search and could in principle climb the cliff — though both reintroduce a form of process supervision, so this would refine rather than overturn the claim. We do not isolate the recurrence from the attention pathway (results are for the hybrid). Internalization is seed-fragile, so per-seed convergence counts, not means, carry the claims. The “process supervision” we dial uses the oracle’s exact latent state, which a real agentic setting lacks — that gap is the point, not a confound. The §6 fixes are tested on the same instrument and do not yet establish transfer to natural workloads.

10. Conclusion

On a single oracle-validated instrument, the composition of state-tracking and recall reduces to one hard sub-problem: learning to maintain non-abelian latent state. Recall — parametric or in-context — is free given a resolved pointer; the wall is the state leg, it requires near-dense process supervision to form, and the internalized circuit that would let an agent run without a scratchpad does not generalize past its trained horizon at any scale up to 357M — the largest fully-converged model still floors. The composition gap is therefore a *learnability* phenomenon rather than an expressivity or capacity one within reach — the empirical content that architectural theses about state tracking leave open. It is also not immovable, but the lever is the *data*, not the model: a sufficient density of target-length training examples unlocks extrapolation where capacity cannot, and a horizon-extension curriculum lifts internalized 4× accuracy from floor to 0.94 in-range (with preliminary, $n = 3$ evidence of partial extrapolation beyond training), while a compressed token-level state summary does not. The difference is that writing the state is itself the step the model cannot yet do unsupervised, so the constructive lesson is to extend the latent recurrence’s trained horizon, not to externalize its state. How far that curriculum reaches, and whether it survives on natural workloads without an oracle to grow the horizon against, is the next question.

References

Draft bibliography — venues are as published; arXiv identifiers are given where verified. Verify all entries (IDs, author lists, pages) before submission.

- Barber, I. (2026). *FactWorld: An Oracle-Validated Instrument for Composing Recall, State-Tracking, and Knowledge*. Companion paper. github.com/ianbarber/factworld.
- Biran, E., Gottesman, D., Yang, S., Geva, M., & Globerson, A. (2024). Hopping too late: exploring the limitations of large language models on multi-hop queries. *EMNLP 2024*.
- Buitrago Ruiz, R., & Gu, A. (2025). Understanding and improving length generalization in recurrent models. *arXiv:2507.02782*.
- Deng, Y., Choi, Y., & Shieber, S. (2024). From explicit CoT to implicit CoT: learning to internalize chain-of-thought step by step. *arXiv*.
- Geva, M., Schuster, R., Berant, J., & Levy, O. (2021). Transformer feed-forward layers are key-value memories. *EMNLP 2021*.
- Grazi, R., Siems, J., Zela, A., Franke, J. K. H., Hutter, F., & Pontil, M. (2024). Unlocking state-tracking in linear RNNs through negative eigenvalues. *arXiv:2411.12537*.
- Liu, B., Ash, J. T., Goel, S., Krishnamurthy, A., & Zhang, C. (2023). Transformers learn shortcuts to automata. *ICLR 2023*; *arXiv:2210.10749*.
- Meng, K., Bau, D., Andonian, A., & Belinkov, Y. (2022). Locating and editing factual associations in GPT. *NeurIPS 2022*.

- Merrill, W., & Sabharwal, A. (2023). The parallelism tradeoff: limitations of log-precision transformers. *TACL*.
- Merrill, W., & Sabharwal, A. (2025). A little depth goes a long way: the expressive power of log-depth transformers. *arXiv:2503.03961*.
- Mozer, M. C., Siddiqui, S. A., & Liu, R. (2026). The topological trouble with transformers. *arXiv:2604.17121*.
- Shao, Z., Wang, P., Zhu, Q., et al. (2024). DeepSeekMath: pushing the limits of mathematical reasoning in open language models. *arXiv* (introduces GRPO).
- Siems, J., Carstensen, T., Zela, A., Hutter, F., Pontil, M., & Grazzi, R. (2025). DeltaProduct: improving state-tracking in linear RNNs via Householder products. *NeurIPS 2025*.
- Yu, Z., Belinkov, Y., & Ananiadou, S. (2025). Back attention: understanding and enhancing multi-hop reasoning in large language models. *EMNLP 2025*.
- Meta FAIR CodeGen Team (2025). CWM: an open-weights LLM for research on code generation with world models. *arXiv:2510.02387*.